

Best Practices for Architecting a Serverless System



Serverless architecture is becoming an exciting option for many cloud architects, considering its unique benefits. Also known as Function-as-a-Service, serverless architecture enables developers to deploy their cloud applications without having to manage any servers. A few best practices that will help organisations benefit from this architecture are discussed in this article.

As of now, there aren't any defined rules to follow when it comes to developing a serverless system. There has been a lot of debate on the best practices, though there are very few of these that are widely accepted. And not having predefined practices makes it hard for organisations to leverage the exact benefits that serverless architectures promise.

If you are building an application keeping in mind scalability, performance

and security, you will be required to follow some of the common serverless practices. Let's discuss them in this article.

Develop a single-purpose stateless function

By default, the recommended practice is to write single-purpose code for the function, since functions are stateless and exist only for a limited period of time. By doing this, you are limiting the execution time, which is directly proportional to the overall costing.

Also, functions with single-purpose code are easier to test, deploy and release. This also improves software agility. With multi-purpose functions running your entire application, the problem arises when scaling up – you end up scaling an entire application rather than just a particular functionality.

For example, let's assume your application has a function that serves two applications, where the first one serves thousands of requests while the second one serves hundreds of requests. Now imagine a situation where you're required to optimise this function. You will have to do it keeping in mind the requirements of both the apps. So split the function and make a separate one for both the applications to simplify the process.

Create a powerful front-end

A poorly conceived front-end requires a lot of back-and-forth between the front-end and back-end. Due to this, processes start lagging, leading to increased latency. To tackle this, the best practice would be to separate the front-end from the back-end.

Executing a more complex business functionality at the front-end, especially through a rich client-side application framework, can potentially help in reducing the cost by minimising the function calls and execution times. The best way to do this would be to completely decouple the back-end logic from the front-end while not compromising on security. At the same time, this allows more services to be accessed from the front-end. This will result in better application performance and a richer user experience.

Design trigger based event-driven patterns

As discussed in the above point, developing a decoupled application can go a long way in terms of scalability. Apart from that, it can help you reduce the interdependencies within functions so that failures do not impact other components, and in enabling